Here is a short introductory paragraph for **Eneo** based entirely on your vision:

Eneo is a house hunting web application built specifically for renting. It strips away unnecessary complexity — no separate tenant and landlord roles, no expensive API keys, and no hidden costs. Landlords post properties by taking photos on-site (with automatic GPS coordinate capture or simple manual fallback), and the app compresses every image to ~200KB using the browser's own Canvas API before storing it on Cloudflare R2's free tier. Tenants search a feed of property cards, view locations with a single click that opens Google Maps with a pre-pinned red marker, and book viewings directly. Bookings appear in the landlord's posting mode alongside the tenant's phone number — communication happens via a simple call or SMS, no chat APIs or WhatsApp integrations. A 24-hour auto-removal rule keeps listings fresh. Everything runs on a static single-page application architecture powered by Supabase's free PostgreSQL tier. Eneo is designed to work, to scale at no cost for as long as possible, and to do exactly what it promises: help renters find their place.

PROJECT BLUEPRINT: House Hunting Web Application (Rental Focus)

PART ONE: OVERVIEW & LOCATION SYSTEM

---

1. PROJECT OVERVIEW

Application Type: Web application for house hunting, primarily for renting purposes.

Core Philosophy:
- Use free tiers and no-cost solutions wherever possible
- Avoid unnecessary API complexity
- Client-side processing preferred
- Simple communication (phone calls/SMS, not API integrations)
- Single codebase for all users (no separate tenant vs landlord roles)

User Model:
- Every user can both rent and post properties
- No separate "tenant" and "landlord" roles

Architecture Type: Single-page application (SPA)
- Everyone loads the same static HTML/CSS/JS shell from the server
- Empty div containers get dynamically filled with cards based on database data
- Uses History API for clean URLs (back button works, links shareable)

---

2. LOCATION SYSTEM

2.1 Capturing Property Coordinates During Posting

Primary Method (GPS):

- When a landlord posts a property and begins taking photos, the website asks: "Can we see your location?"
- If accepted and no VPN is detected, the browser's geolocation reads the current position
- Latitude and longitude are stored in the database

Pseudocode for GPS capture:
```
if (geolocation available in browser) {
  ask user permission
  if (permission granted) {
    get current position (lat, lng)
    store in database with property listing
  }
}
```

2.2 Manual Coordinate Entry (Fallback Method)

When used:
- User is far from the property
- User is using a VPN
- GPS permission denied or inaccurate

Process:
1. User clicks a manual entry option
2. A short animation/tutorial plays showing the exact steps:
   - Open Google Maps (web or app)
   - Right-click (desktop) or long-press (mobile) on the property location
   - Click "What's here?" (desktop) or swipe up (mobile)
   - Copy the numbers shown (e.g., -1.286389, 36.817223)
   - Paste into the website input field
3. User pastes coordinates in format: `latitude, longitude`
4. Website validates format and saves to database

Pseudocode for manual entry:
```
function showCoordinatesTutorial():
  display modal with step-by-step animation
  provide input field for pasting
  user pastes coordinates
  validate format: two numbers separated by comma
  if valid:
    save to database
  else:
    show error: "Invalid format. Use: latitude,longitude"
```

2.3 Displaying Properties on Map (Tenant View)

Method: Google Maps URL scheme (no API key required)

Process:

- When a tenant clicks the map section or a "View on Map" button
- The website constructs a URL using the stored coordinates:

```
https://www.google.com/maps/search/?api=1&query={latitude},{longitude}
```

- This URL opens Google Maps (or Google Maps app on mobile) with a red pin/drop marker at the exact property location

Pseudocode:
```
function openGoogleMaps(lat, lng):
  url = "https://www.google.com/maps/search/?api=1&query=" + lat + "," + lng
  window.open(url, '_blank')
```

Important notes:
- No Google Maps API key required
- No egress costs
- Works on both desktop and mobile
- Mobile opens the Google Maps app if installed

END OF PART ONE

Confirm you have received this and are ready for **Part Two** (Image Handling, Database, and Cloud Storage).

PROJECT BLUEPRINT: House Hunting Web Application (Rental Focus)

PART TWO: IMAGE HANDLING, DATABASE & CLOUD STORAGE

---

3. IMAGE HANDLING SYSTEM

3.1 Client-Side Compression (Before Upload)

Trigger: User selects photo from gallery or takes photo with camera

Process:
- Script runs entirely in the browser (no server processing)
- Image is compressed to approximately 200KB
- EXIF data (GPS coordinates, camera model, timestamps) is stripped automatically for privacy

Tools Used: Canvas API (built into browser, completely free)

Compression Parameters:

- Target file size: ~200KB
- Max dimension: 1200px (width or height, whichever is larger)
- Format: JPEG
- Quality setting: 0.7 (on a 0-1 scale)

Memory Safety Checks:
- Before processing, check if image exceeds approximately 12 megapixels (e.g., 4000×3000)
- If too large, show error: "Image too large. Please use a smaller photo or take a normal camera photo"
- This prevents crashing low-memory mobile devices

Pseudocode for compression:
```
function compressImage(originalFile, maxSizeKB = 200):
  return new Promise:
    read file as data URL
    create new Image object
    when image loads:
      calculate target dimensions
      maxDimension = 1200
      if width > maxDimension or height > maxDimension:
        scale down proportionally to fit within maxDimension

      create canvas element with target width/height
      draw image onto canvas

      convert canvas to blob (JPEG format, quality 0.7)

      return compressed blob as File object
```

User Flow for Upload:
1. User selects photo from gallery or takes new photo
2. JavaScript reads file (still in browser, not uploaded yet)
3. Canvas resizes to max 1200px dimension
4. Quality set to 0.7 (JPEG)
5. Resulting file: ~150-250KB
6. Preview thumbnail shown on screen
7. User clicks "Upload" button
8. Upload begins (takes 1-2 seconds)
9. Cloudflare R2 returns permanent URL
10. URL stored in database

Why Canvas API is free: It is built into every web browser as a standard feature, like `<video>` tags. No payment, no signup, no usage limits.

Note: Canvas API is different from "ChatGPT Canvas" (OpenAI product) and "Canva" (design platform). This is the browser's built-in Canvas API.

3.2 EXIF Data Stripping

Method: Using `canvas.toBlob()` automatically creates a brand new image file from scratch. The original file with EXIF data is discarded. This automatically strips:
- GPS coordinates (privacy protection for landlords)
- Camera make and model
- Timestamps
- Other metadata

No additional code needed — the compression process handles this automatically.

3.3 Cloud Storage for Photos

Service: Cloudflare R2

Why chosen:
- Free tier: 10GB storage
- Zero egress fees (no charge when users view images)
- 1 million write/delete operations per month free

What is stored: Only the compressed images (~200KB each)

What is NOT stored: Original large files, raw photos, EXIF data

Storage Estimate:
- Average property: ~20 photos (4MB total per property at 200KB each)
- Free tier capacity: Approximately 2,500 properties before reaching 10GB limit

URL Format: Cloudflare R2 returns a permanent public URL for each uploaded image

3.4 Database Image Storage

Method: Only store the Cloudflare R2 URLs in the database, not the images themselves

Format: JSON array of URL strings

Example database field:
```
photos: ["https://pub-xxx.r2.dev/photo-123.jpg", "https://pub-xxx.r2.dev/photo-456.jpg"]
```

Size per property: Approximately 1-2KB for the JSON array (URLs only)

Why this matters:
- Keeps database small (500MB Supabase free tier goes far)
- Images load directly from Cloudflare to tenant's browser
- No bandwidth cost from database on image views

3.5 Image Loading for Tenants

Method: Lazy loading

Process:
- Website loads the JSON array of URLs from database (small, ~1-2KB)

- Browser downloads images directly from Cloudflare R2 URLs
- Images load progressively as tenant scrolls

Bandwidth: Images served by Cloudflare R2 (zero egress fees on their free tier)

---

4. DATABASE SYSTEM

4.1 Database Technology

Choice: PostgreSQL via Supabase

Why chosen:
- Free tier: 500MB database storage
- 5GB bandwidth per month (separate from storage)
- Built-in authentication (user login/signup)
- Real-time updates available
- Web interface for viewing data (like Excel)
- Auto-generated API

Free Tier Limitations:
- Database storage: 500MB (service restricted if exceeded)
- Bandwidth (egress): 5GB total (service restricted if exceeded on free tier)
- File storage: 1GB (separate from database storage)
- Compute: Shared, pauses after 1 week of inactivity

Note on bandwidth: Bandwidth limit is the more likely constraint before storage limit. Approximately 100,000 API calls per month at 50KB per response.

When to upgrade: Approximately when exceeding 500 daily active users making frequent searches. Pro plan is $25/month.

Free tier never expires: Unlike some services, Supabase free tier is not a 12-month trial.

4.2 Database Tables

Properties Table:
Stores property listings posted by users

Fields required:
- id (primary key, auto-generated)
- landlord_id (references users table)
- title
- description
- property_type (apartment, house, condo, studio, townhouse)
- bedrooms (integer)
- bathrooms (decimal, e.g., 1.5)
- price (monthly rent)
- latitude (decimal, up to 10 digits with 8 decimal places)
- longitude (decimal, up to 11 digits with 8 decimal places)
- photos (JSON array of Cloudflare R2 URLs)

- status (active, removed, draft)
- created_at (timestamp)

Bookings Table:
Stores booking requests from tenants to landlords

Fields required:
- id (primary key)
- property_id (references properties table)
- tenant_id (references users table)
- booked_at (timestamp)
- status (pending, confirmed, expired)

Users Table:
Stores all user accounts

Fields required:
- id (primary key)
- email
- hashed_password
- name
- phone (optional — shown to tenants who book)
- account_id (unique, immutable identifier, format example: "usr_abc123xyz")
- created_at (timestamp)

4.3 Spatial Queries (Finding Nearby Properties)

When tenant opens feed mode, find properties within a radius:

Approach: Use distance calculation formula that works with latitude/longitude

Pseudocode for nearby properties query:
```
SELECT *,
  (6371 * acos(cos(radians(user_latitude)) * cos(radians(latitude)) *
    cos(radians(longitude) - radians(user_longitude)) +
    sin(radians(user_latitude)) * sin(radians(latitude)))) AS distance_km
FROM properties
WHERE status = 'active'
HAVING distance_km < 5
ORDER BY distance_km
```

Notes:
- 6371 is Earth's radius in kilometers (for km results)
- For better performance on large datasets, spatial indexing would be needed
- Within free tier limits, this works adequately for hundreds/thousands of listings

---

END OF PART TWO

Ready for **Part Three** (Communication, Booking Flow, Auto-removal, and Three Modes Overview) when you say.

PROJECT BLUEPRINT: House Hunting Web Application (Rental Focus)

PART THREE: COMMUNICATION, BOOKING FLOW, AUTO-REMOVAL & THREE MODES

---

5. COMMUNICATION SYSTEM

5.1 Contact Method

Choice: Phone number only (no WhatsApp API, no in-app messaging)

Why chosen:
- No business verification required
- No message template approval process
- No per-conversation costs
- No rate limits or throttling
- Works on any phone (does not require WhatsApp installed)
- No debugging of failed messages
- Simpler to implement

Display on property page (after booking):
- Landlord's phone number shown
- "Call Now" button with tel: link
- "Send SMS" button with sms: link
- Optional: Best time to call (e.g., 9 AM - 6 PM)

Pseudocode for contact display:
```
<div class="contact-section">
  <h3>Contact Landlord</h3>
  <p class="phone-number">+254 712 345 678</p>
  <button onclick="window.location.href='tel:+254712345678'">
    Call Now
  </button>
  <button onclick="window.location.href='sms:+254712345678'">
    Send SMS
  </button>
</div>
```

Note: The `tel:` link works on all phones — one tap and they are calling.

5.2 What is NOT Included

- No WhatsApp API integration
- No in-app chat system
- No email-based communication (beyond what may already exist separately)

---

6. BOOKING FLOW

6.1 Tenant Booking Process

Steps:

1. Tenant finds property in feed mode
2. Clicks on property card to see full details (photos, map, description)
3. Clicks "Book This Property" button
4. System creates booking record in database with:
   - Property ID
   - Tenant ID
   - Current timestamp
   - Status: "pending"

5. Confirmation shown to tenant: "Booking requested! Landlord will see this in their posting mode."

Pseudocode:
```
function bookProperty(propertyId):
  insert into bookings table:
    property_id = propertyId
    tenant_id = currentUserId
    booked_at = current timestamp
    status = 'pending'

  show alert: "Booking requested! Landlord will see this in their posting mode."
```

6.2 Landlord Receiving Booking Notifications

How landlord sees bookings:
- Landlord navigates to Posting Mode
- In their Active Listings section, each property shows its bookings
- Bookings display: tenant name, booked time, and a "Call" button

No automatic notification via WhatsApp/email/SMS — landlord must check Posting Mode

Pseudocode for displaying bookings to landlord:
```
for each property owned by current user:
  fetch bookings where property_id = property.id and status = 'pending'

  for each booking:
    display: "[Tenant Name] booked on [booked_at]"
    display button: "Call [Phone Number]"
```

```

#### 6.3 Landlord Actions on Bookings

**Landlord's responsibilities:**
- See who booked the property (tenant name and phone number)
- Call the tenant directly using the displayed phone number
- Arrange viewing time via phone call

**No in-app confirmation button** — conversation happens via phone call

---

### 7. AUTO-REMOVAL OF BOOKED PROPERTIES

#### 7.1 The 24-Hour Rule

**Rule:** When a property receives a booking, it is automatically taken down after 24 hours if no action is taken by the landlord.

**Purpose:** Ensures landlords are responsive and properties are not left in "booked" state indefinitely.

#### 7.2 How It Works

**Process:**
- Booking is created with timestamp
- System checks (via database trigger or scheduled job) for bookings older than 24 hours
- If property has a booking older than 24 hours and status is still "pending":
  - Property status changes from "active" to "removed"
  - Removal reason recorded: "24hr booking no-contact"

**Pseudocode for auto-removal check:**
```
function checkExpiredBookings():
  find all bookings where:
    booked_at < (current time - 24 hours)
    status = 'pending'

  for each such booking:
    update properties table:
      set status = 'removed'
      set removed_reason = '24hr booking no-contact'
      where id = booking.property_id
```

#### 7.3 Landlord Manual Removal

**Alternative:** Landlord can manually take down a property at any time

**Method:** Button in Posting Mode labeled "Take Down"
```

Effect: Property status changes from "active" to "removed" instantly (no 24-hour wait)

Pseudocode:
```
function takeDownProperty(propertyId):
  update properties table:
    set status = 'removed'
    set removed_reason = 'landlord manually removed'
    where id = propertyId

  show alert: "Property has been taken down."
```

---

8. THE THREE MODES

The application has three main modes that users can switch between.

Mode 1: Feed Mode (Default view after login)

What it shows:
- Properties near the user (based on GPS or saved location)
- Search bar for finding specific properties
- Filters for refining results (price range, number of bedrooms, property type)
- Infinite scroll of property cards

Card design:
- Few pictures for first impression
- Summary information:
  - Number of bedrooms
  - Number of bathrooms
  - Price per month
  - Location description (e.g., "200m from Sarit Centre")
  - Badges (pets allowed, etc.)

Interaction:
- User swipes up to see more cards (like Facebook feed)
- User clicks on a card to see full property details

Full property detail page (after clicking card):
- All photos (full size gallery)
- Maps information (button to open Google Maps with pin)
- Full description
- "Book This Property" button (triggers booking flow)
- Contact information shown only after booking (or on booking confirmation?)

Note from discussion: Contact information timing not fully specified — implement per best judgment.

Feed source: Properties from database where status = 'active', ordered by creation date or proximity.

Mode 2: Posting Mode

Purpose: For users to post their own properties and track their postings.

Sections within Posting Mode:

A. Active Listings
- Properties currently posted and visible in Feed Mode
- For each property display:
  - Thumbnail image
  - Title
  - Date posted
  - Number of views (if tracked)
  - Number of bookings received
  - List of bookings showing:
    - Tenant name
    - Time of booking
    - Button to call the tenant
  - "Edit" button (to modify property details)
  - "Take Down" button (removes listing immediately)

B. Drafts / Pending (Properties saved for later review)
- Properties that have been started but not yet published
- Landlord can edit and publish, or delete

C. Recently Removed
- Properties that have been taken down (manually or by 24-hour rule)
- Landlord can restore within 30 days (implied — not explicitly stated but logical extension of "saved for later review")

Posting a new property:
- Form includes: title, description, property type, bedrooms, bathrooms, price
- Photo upload (with client-side compression)
- Location capture (GPS or manual coordinate entry)
- Submit button creates new record in properties table with status = 'active'

Tracking bookings:
- Landlord can see which properties got booked
- Landlord can see who booked them (name, phone number)
- Landlord can call tenants directly from this mode

Mode 3: Account Mode

Purpose: User account management

Features included:
- Login
- Password management (change password)
- Account name management
- Read-only Account ID (assigned at signup, cannot be changed)
- Format example: "usr_abc123xyz"

Features NOT explicitly discussed:
- Delete account (mentioned but not specified as yes/no — include or omit per discretion)
- Email change (not discussed)
- Phone number change (not discussed)

Pseudocode for Account Mode display:
```
<div class="account-page">
  <form>
    <label>Name</label>
    <input type="text" value="{user.name}">

    <label>Email</label>
    <input type="email" value="{user.email}">

    <label>Phone (optional — for tenant to call you)</label>
    <input type="tel" value="{user.phone}">

    <button>Change Password</button>
  </form>

  <div>
    Your Account ID: {user.account_id}
    (Cannot change — this is your unique identifier)
  </div>
</div>
```

---

9. MODE SWITCHING (Navigation)

User can switch between three modes at any time after login:
- Feed Mode
- Posting Mode
- Account Mode

Implementation approach: Tab navigation or bottom navigation bar (not specified — implement per best judgment).

END OF PART THREE

Ready for **Part Four** (Data Flow, Frontend Architecture, and Complete System Summary) when you say.

PROJECT BLUEPRINT: House Hunting Web Application (Rental Focus)

PART FOUR: DATA FLOW, FRONTEND ARCHITECTURE & COMPLETE SYSTEM SUMMARY

---

10. DATA FLOW ARCHITECTURE

10.1 Single-Page Application (SPA) Data Flow

Core Principle: Everyone loads the same static website from the server. The website contains empty HTML div containers. Data from the database fills these containers dynamically.

Step-by-step flow:

1. User visits website URL
2. Server returns static HTML/CSS/JS files (same for all users)
3. Browser loads and displays login/signup screen
4. After successful login, application knows who the user is
5. Application fetches user-specific data from Supabase database
6. Application dynamically renders property cards into empty divs
7. User interacts (clicks, searches, filters) → new data fetched → divs re-render

Pseudocode for initial load:
```
on page load:
  display login screen

on successful login:
  store current user ID in application state
  loadFeedMode()  // default view

function loadFeedMode():
  clear main container div
  fetch properties from Supabase where status = 'active'
  for each property:
    create property card HTML
    append to container div
```

10.2 Empty Divs Structure

The static HTML contains empty container divs that will be filled with content:

```html
<div id="feed-container"></div>
<div id="posting-container" style="display:none;"></div>
<div id="account-container" style="display:none;"></div>
```

Only one container is visible at a time. Which container is visible depends on which mode the user has selected.

Mode switching pseudocode:
```
function showFeedMode():
  hide posting-container
  hide account-container
  show feed-container
  loadFeedMode()  // fetch and render properties

function showPostingMode():
  hide feed-container
  hide account-container
  show posting-container
  loadPostingMode()  // fetch user's listings and bookings

function showAccountMode():
  hide feed-container
  hide posting-container
  show account-container
  loadAccountMode()  // fetch user's profile data
```

10.3 Data Fetching Pattern

For each mode, the application makes Supabase API calls to fetch only the data needed for that view.

Feed Mode data fetch:
```
fetch from Supabase:
  table: properties
  where: status = 'active'
  order: created_at descending
  limit: 20 (initial load, load more on scroll)
```

Posting Mode data fetch:
```
fetch from Supabase:
  table: properties
  where: landlord_id = currentUserId

also fetch from Supabase:
  table: bookings
  where: property_id IN (list of user's property IDs)
  join with users table to get tenant names and phone numbers
```

Account Mode data fetch:
```
fetch from Supabase:
  table: users
  where: id = currentUserId
```

```
  fields: name, email, phone, account_id
```

10.4 Dynamic Card Rendering

Feed Mode Property Card structure:
Each card contains:
- Image gallery (first few photos from the photos JSON array)
- Number of bedrooms
- Number of bathrooms
- Price per month
- Location description
- Badges (pet policy, etc.)
- "Click to see more" button

Pseudocode for card creation:
```
function createPropertyCard(property):
  cardDiv = create div with class "property-card"
  cardDiv.dataset.propertyId = property.id

  imageGallery = create div with class "image-gallery"
  firstImage = create img element with src = property.photos[0]
  photoCount = create div with "📷 {property.photos.length} photos"
  imageGallery.appendChild(firstImage)
  imageGallery.appendChild(photoCount)

  summaryDiv = create div with class "card-summary"
  title = create h3 with property.title
  price = create p with "KSh {property.price}/month"
  bedrooms = create span with "🏠 {property.bedrooms} bedrooms"
  bathrooms = create span with "🛁 {property.bathrooms} bath"

  detailsButton = create button with "Click to see more →"
  detailsButton.onclick = () => showPropertyDetail(property.id)

  cardDiv.appendChild(imageGallery)
  cardDiv.appendChild(summaryDiv)
  cardDiv.appendChild(detailsButton)

  return cardDiv
```

10.5 Property Detail View (After Clicking Card)

When user clicks "Click to see more" button:

1. Fetch full property data from database
2. Hide feed container
3. Show detail container (or modal)
4. Render:
   - All photos from photos JSON array (full size gallery)

- "Open in Google Maps" button using coordinates
- Full description text
- "Book This Property" button
- (Contact information displayed after booking per earlier specification)

Pseudocode for detail view:
```
function showPropertyDetail(propertyId):
  fetch property from Supabase where id = propertyId

  clear detail container
  create image gallery with all property.photos
  create map button:
    onclick = openGoogleMaps(property.latitude, property.longitude)
    text = "📍 Open in Google Maps"
  create description paragraph
  create book button:
    onclick = () => bookProperty(propertyId)

  show detail container
```

Google Maps function (repeated from Part One):
```
function openGoogleMaps(lat, lng):
  url = "https://www.google.com/maps/search/?api=1&query=" + lat + "," + lng
  window.open(url, '_blank')
```

10.6 Posting Mode Rendering

Active Listings section:
For each property owned by user:
- Property thumbnail (first photo)
- Property title
- Posted date
- View count (if implemented)
- Number of bookings
- List of bookings (tenant name, booked time, call button)
- Edit button
- Take Down button

Post new property form:
- Text inputs: title, description
- Select inputs: property type, bedrooms, bathrooms
- Number input: price
- File input: photos (triggers compression on selection)
- Location capture button (triggers GPS or manual entry flow)
- Submit button

---

11. URL STRUCTURE (Even Though SPA)

Even though the application is a single-page app, clean URLs can be implemented using the History API. This makes the back button work and allows link sharing.

Desired URL mapping:
- `/feed` → Feed Mode
- `/property/{id}` → Property detail view for property ID
- `/my-listings` → Posting Mode
- `/account` → Account Mode

Pseudocode for URL handling:
```
function navigateToFeed():
  history.pushState({ mode: 'feed' }, '', '/feed')
  showFeedMode()

function navigateToProperty(propertyId):
  history.pushState({ mode: 'detail', id: propertyId }, '', '/property/' + propertyId)
  showPropertyDetail(propertyId)

function navigateToPosting():
  history.pushState({ mode: 'posting' }, '', '/my-listings')
  showPostingMode()

function navigateToAccount():
  history.pushState({ mode: 'account' }, '', '/account')
  showAccountMode()

// Handle browser back button
window.onpopstate = function(event):
  if event.state.mode == 'feed':
    showFeedMode()
  else if event.state.mode == 'detail':
    showPropertyDetail(event.state.id)
  else if event.state.mode == 'posting':
    showPostingMode()
  else if event.state.mode == 'account':
    showAccountMode()
```

---

12. COMPLETE SYSTEM SUMMARY

12.1 Technology Stack

Component	Technology	Cost
Frontend hosting	Static web server	Unknown (not specified)
Database	PostgreSQL via Supabase	Free tier (500MB storage, 5GB bandwidth)
Image storage	Cloudflare R2	Free tier (10GB storage, zero egress fees)

Image compression	Browser Canvas API	Free
Maps	Google Maps URL scheme	Free (no API key)
Communication	Phone calls/SMS	Standard carrier rates

12.2 Complete User Flows

Flow A: Tenant finds and books a property
1. User logs into account
2. Feed Mode loads, shows property cards near user
3. User swipes/filters to find property of interest
4. User clicks property card
5. Full property detail view opens (photos, map button, description)
6. User clicks "Book This Property" button
7. System creates booking record in database
8. User sees confirmation message

Flow B: Landlord receives and responds to booking
1. Landlord logs into account
2. Landlord switches to Posting Mode
3. Active Listings section shows properties with pending bookings
4. Landlord sees: "Jane M. booked today at 2 PM"
5. Landlord clicks "Call" button next to booking
6. Phone app opens with tenant's number
7. Landlord calls tenant to arrange viewing

Flow C: Landlord posts a new property
1. Landlord switches to Posting Mode
2. Clicks "Post New Property" button
3. Fills out form (title, description, type, bedrooms, bathrooms, price)
4. Takes or selects photos from phone
5. Photos are automatically compressed to ~200KB
6. Website asks for location permission
7. If granted → GPS coordinates captured
8. If denied/error → manual coordinate entry tutorial shown
9. User pastes coordinates from Google Maps
10. Clicks "Submit"
11. Property saved to database with status 'active'
12. Property immediately appears in Feed Mode for tenants

Flow D: Property auto-removal after booking
1. Tenant books a property at time T
2. Landlord does not call tenant within 24 hours
3. System detects booking older than 24 hours with status 'pending'
4. Property status changed from 'active' to 'removed'
5. Property no longer appears in Feed Mode
6. Landlord sees property in "Recently Removed" section with reason: "24hr booking no-contact"

Flow E: Landlord manually takes down property
1. Landlord views active listing in Posting Mode
2. Clicks "Take Down" button on a property
3. Property status changed from 'active' to 'removed' instantly
4. Property no longer appears in Feed Mode

5. Landlord can restore from "Recently Removed" section (implied)

12.3 Free Tier Limitations (Summary)

Service	Limit	What happens at limit
Supabase database	500MB	Service restricted
Supabase bandwidth	5GB/month	Service restricted
Cloudflare R2 storage	10GB	Overages billed or restricted
Cloudflare R2 operations	1M write/delete	Overages billed
Canvas API	No limits	Runs on user device
Google Maps URLs	No limits	No cost

12.4 Features Explicitly NOT Included

- WhatsApp API integration
- In-app chat/messaging system
- Google Maps API (using URL scheme instead)
- Separate tenant vs landlord roles (all users can do both)
- Automatic notifications via SMS/email
- Payment processing
- Lease agreement signing
- Tenant screening/credit checks
- Landlord verification badges (other than account ID)

12.5 Features Included as Discussed

- GPS coordinate capture during property posting
- Manual coordinate entry with tutorial animation
- Google Maps pins via URL scheme
- Client-side image compression to ~200KB using Canvas API
- EXIF data stripping automatically
- Cloudflare R2 for image storage
- Supabase PostgreSQL database
- Three modes: Feed, Posting, Account
- Booking system with 24-hour auto-removal
- Phone number-only communication
- Single-page application architecture
- Dynamic card rendering from empty divs
- Clean URLs with History API

---

13. DOCUMENT NOTES

This document contains only features explicitly discussed in the conversation. No additional features, design choices, or implementation details have been added beyond what was mentioned.

Items not specified (to be determined during implementation):
- Color scheme
- Typography
- Exact button placements

- Loading states
- Error message wording (beyond examples provided)
- Responsive breakpoints
- Logo/branding
- Domain name
- Hosting platform for static files
- Date/time format preferences
- Currency symbol (KSh used as example from Nairobi context)
- Whether view counts are tracked
- Whether properties can be edited after posting
- Whether deleted properties can be restored
- Maximum number of photos per property
- Whether users can delete their account
- Email verification during signup
- Password reset flow
- Session duration (how long login lasts)

These items should be decided during the construction phase based on developer judgment.

END OF PART FOUR
END OF COMPLETE BLUEPRINT DOCUMENT

You now have the complete four-part blueprint document covering everything discussed. Each part can be used as a reference during construction. No additions, no omissions — just the features, flows, and architecture we walked through together.

NB NOT ACTUAL DESCRIPTION JUST INSPIRATIONAL LINES U CAN WORK WITH

Understood. You want me to:

1. **Focus on the Posting Mode form** (the card/form that appears when user taps "+ New Post") — describe how it looks and works in detail

2. **Modify the Filters** — remove Property Type and Pet Policy, keep Price Range, Bedrooms, Bathrooms only

3. Provide only the modified sections (not the whole UI again)

Here we go.

MODIFIED FILTER SECTION (Feed Mode)

When user taps the filter icon (slider icon) in Feed Mode, a bottom sheet slides up.

What the user sees inside the filter sheet:

- Title at top: "Filters"

- **Price Range** — two text input fields side by side:
  - Left field: "Min (KSh)" with gray placeholder text
  - Right field: "Max (KSh)" with gray placeholder text
  - Below these fields, small gray helper text: "Leave blank for no limit"

- **Bedrooms** — a row of chip buttons horizontally scrollable if needed:
  - "Any" (active by default)
  - "1"
  - "2"
  - "3"
  - "4+"
  - Only one chip can be active at a time. "Any" means no bedroom filter.

- **Bathrooms** — a row of chip buttons:
  - "Any" (active by default)
  - "1"
  - "1.5"
  - "2"
  - "3+"
  - Only one chip active at a time.

- At the bottom of the sheet, two buttons side by side:
  - Left button: "Clear All" — gray outline, white inside, gray text
  - Right button: "Apply Filters" — solid green, white text

Interaction:

User sets price range (e.g., Min 20000, Max 50000). User selects bedrooms "2" and bathrooms "1.5".

User taps "Apply Filters." Sheet slides down. Feed reloads showing only properties matching:
- Price between KSh 20,000 and KSh 50,000
- Exactly 2 bedrooms
- Exactly 1.5 bathrooms

If user taps "Clear All":
- Price Min and Max fields become empty
- Bedrooms chip resets to "Any"
- Bathrooms chip resets to "Any"
- Sheet stays open

POSTING MODE FORM — COMPLETE DESCRIPTION

How the user gets there:

User is in Posting Mode (📝 icon at bottom). At top-right of screen, a green button with a plus icon: "+ New Post". User taps it.

What happens next:

The screen transitions. The Posting Mode screen slides to the left. A new screen slides in from the right — the **New Posting Form**.

New Posting Form — Screen Layout

Top of screen (header bar):
- Left side: A back arrow icon (←) in green
- Center: "New Listing" in bold dark text
- Right side: Nothing (empty)

Below the header — scrolling form content:

Section 1: Basic Information

- **Title** — A text input field. Gray placeholder text: "e.g., Cozy 2BR Apartment in Westlands"
  - Below the field, small gray helper text: "This is the first thing tenants will see"

- **Description** — A larger text area (3-4 lines tall). Gray placeholder text: "Describe the property — size, nearby amenities, parking, security features, anything special..."
  - Below the field, small gray helper text: "Be detailed but clear"

Section 2: Property Details

Two fields side by side on the same row (on mobile, they stack vertically on very small screens, but on most phones appear side by side):

- **Bedrooms** — Number input field (with up/down arrows). Placeholder: "0"
  - Small helper text below: "Number of bedrooms"

- **Bathrooms** — Number input field with decimal support (step="0.5"). Placeholder: "0"
  - Small helper text below: "e.g., 1, 1.5, 2"

- **Price (per month)** — Number input field. Placeholder: "35000"
  - Before the field, a KSh label (Kenyan Shillings)
  - Small helper text below: "Monthly rent in KSh"

Section 3: Photos

A large rounded gray box (about 150px tall) with a dashed border.

Inside the box:
- A large plus icon (+) in the center
- Text below the icon: "Tap to add photos"
- Smaller text under that: "First photo will be the cover image"

When user taps the gray box:
- On mobile: Opens system dialog — "Take Photo" or "Choose from Gallery"
- User selects one or multiple photos (max 20 — number not specified earlier, choosing 20 as reasonable)

After selecting photos:

The gray box changes. Now it shows:

- A horizontal scrolling row of thumbnails (each about 80x80px)
- Each thumbnail shows the compressed photo preview
- At the top-right corner of each thumbnail, a small red circle with a white "X" — user can tap to remove that photo before submitting
- At the end of the thumbnail row, a small green circle with a plus icon — tap to add more photos

While photos are being compressed:
- Each thumbnail shows a small spinner overlay temporarily
- When compression finishes (to ~200KB), the spinner disappears and the thumbnail becomes clear

Below the thumbnail row, small text: "Photos compressed to ~200KB each for fast loading"

Section 4: Location

Title: "Property Location" in bold text.

Two large buttons stacked vertically:

Button A — "📍 Use my current location"
- Green outline, white inside, green text
- Below button: "Easiest if you are at the property"

Button B — "📍 Enter coordinates manually"
- Gray outline, white inside, gray text
- Below button: "Use this if you are far away or using VPN"

Option A — Use my current location (user taps this button):

- Browser immediately asks: "Allow Eneo to access your location?"
- User chooses "Allow"
- A spinner appears inside the button for 1-2 seconds
- If successful:
  - Button changes to solid green with white checkmark icon and text: "✓ Location captured"

  - Below the button, small green text appears: "Latitude: -1.286389, Longitude: 36.817223"
  - A subtle green border appears around this section
- If user denies permission:
  - A red message appears below the button: "Location access denied. Please use manual entry."
  - Button B remains available

- If GPS fails for any other reason (poor signal, VPN detected):
  - Red message: "Could not detect location. Please use manual entry."

Option B — Enter coordinates manually (user taps this button):

A small modal card slides up from the bottom. It has rounded top corners.

Inside the modal:
- Title: "Enter Coordinates"
- A simple illustrated animation showing three steps (icons and short text):
  1. 🌐 Open Google Maps
  2. 📍 Right-click (or long-press) the property location → "What's here?"
  3. 📋 Copy the numbers (e.g., -1.286389, 36.817223)
- A text input field below the animation. Placeholder: "-1.286389, 36.817223"
- A green button: "Save Coordinates"
- A gray text link at bottom: "Cancel"

User pastes coordinates into the field (e.g., "-1.286389, 36.817223").

Validation:
- If format is valid (two numbers separated by comma):
  - A green checkmark appears next to the field
  - Button "Save Coordinates" becomes active (was gray, now green)
- If format is invalid:
  - Red text appears below field: "Invalid format. Use latitude,longitude (e.g., -1.286389, 36.817223)"
  - Button remains gray

User taps "Save Coordinates":
- Modal slides down and closes
- Back on the main form, Button B now shows: "✓ Coordinates saved"
- Below button, small green text shows the actual coordinates

---

Important: Only one location method can be active. If user captures location via GPS, Button B remains available but shows a message "Switch to manual?" — but not explicitly discussed. For simplicity: once coordinates are captured (either method), the user cannot switch unless they tap a "Change location" link. Let's add that:

- After coordinates are captured, a small gray "Change" link appears next to the success message
- User taps "Change" → both buttons reset, user can choose again

---

Section 5: Submit Button

At the bottom of the form, a large sticky green button (fixed at bottom of screen while scrolling):

Text: "POST PROPERTY"

When tapped:

- All fields are validated:
  - Title not empty
  - Description not empty
  - Bedrooms is a number
  - Bathrooms is a number
  - Price is a number greater than 0
  - At least one photo added
  - Coordinates captured (either method)

- If any field is missing:
  - The screen scrolls to the first empty field
  - That field gets a red border
  - A red toast message appears at top: "Please fill in all required fields"

- If all fields valid:
  - Button shows a white spinner inside it (button text becomes hidden, spinner visible)
  - All form fields become disabled (grayed out) to prevent double submission
  - System saves property to database
  - System uploads photos to Cloudflare R2, gets URLs
  - System updates database record with photo URLs
  - Success message appears: "✓ Property posted successfully!"
  - After 1 second, user automatically returns to Posting Mode > Active tab
  - New listing appears at the top of their active listings

If user cancels mid-way:

User taps the back arrow (←) at the top-left of the screen.

A small confirmation bottom sheet appears:

- Title: "Cancel listing?"
- Text: "You have unsaved changes. Do you want to save this as a draft before leaving?"
- Three buttons:
  - "Save as Draft" (green outline)
  - "Discard" (red text)
  - "Cancel" (gray text — stays on form)

If user taps "Save as Draft":
- Current form data is saved to database as draft (status = 'draft', not visible in feed)
- User returns to Posting Mode > Drafts tab
- The draft appears in the Drafts list with thumbnail, title (or "Untitled"), and last edited timestamp

If user taps "Discard":
- Form data is lost
- User returns to Posting Mode > Active tab

If user taps "Cancel":
- Sheet closes, user remains on the form

Edit Draft flow (from Posting Mode > Drafts tab):

User taps "✏️ Edit Draft" on a draft card.

- The same New Posting Form screen opens
- All fields are pre-filled with the saved draft data
- Title bar now says "Edit Listing" instead of "New Listing"
- The submit button text changes from "POST PROPERTY" to "UPDATE LISTING"
- User makes changes, taps "UPDATE LISTING"
- If user taps back arrow, same "Save as Draft / Discard / Cancel" options appear

---

Edit Active Listing flow (from Posting Mode > Active tab):

User taps "✏️ Edit" on an active listing.

- Same form opens, pre-filled with existing data
- Title bar: "Edit Listing"
- Submit button: "UPDATE LISTING"
- After update, user returns to Posting Mode > Active tab with updated listing

Note about photos in edit mode:
- Existing photos are shown as thumbnails in the photo section
- User can tap X on any existing photo to remove it
- User can add new photos (these will be compressed and uploaded)
- Removing a photo deletes it from Cloudflare R2 (or at least removes URL from database)

Summary of changes made:

Section	What changed
Filters	Removed Property Type and Pet Policy. Kept only Price Range, Bedrooms, Bathrooms.
Posting Form	Completely described — every field, interaction, validation, success/error states, draft saving, edit flows.

Do you want me to continue with another part of the UI, or is this sufficient for the Posting Mode form and modified filters?